

The Principles And Practice of Co-design

AI workloads and GPU technology

Alan Smith

Corporate Fellow, Graphics Architecture, AMD

Georgia Tech

ECE/CS 4803/8803

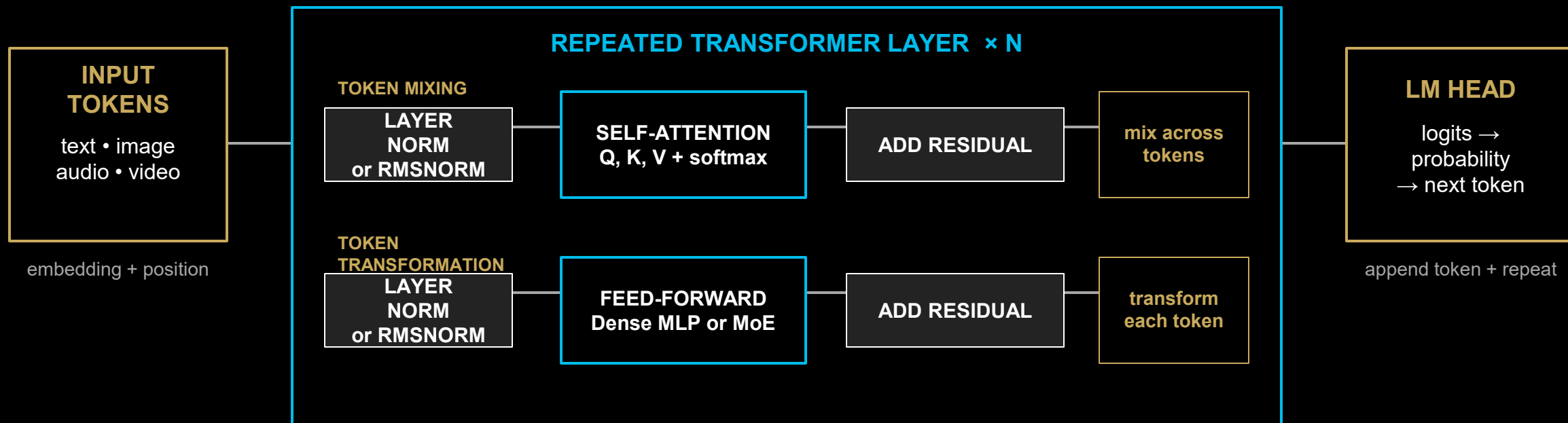
September 9, 2026

The AMD Instinct logo is displayed on a white rectangular background. It features the word "AMD" in a bold, sans-serif font, followed by a square icon containing a stylized, geometric representation of a chip or circuit. Below this, the word "INSTINCT" is written in a smaller, all-caps, sans-serif font.

AMD
INSTINCT

An LLM repeatedly predicts the next token

Large language model: a neural network trained to estimate $p(\text{next token} \mid \text{preceding context})$



ATTENTION

moves information across the context

NORMALIZATION

stabilizes activations through depth

MLP / MoE

holds most parameters and matrix compute

Generation is a feedback loop: select a token, append it to context, and run the stack again

The same LLM creates three very different GPU workloads

Training learns the weights; inference first ingests context, then generates tokens autoregressively

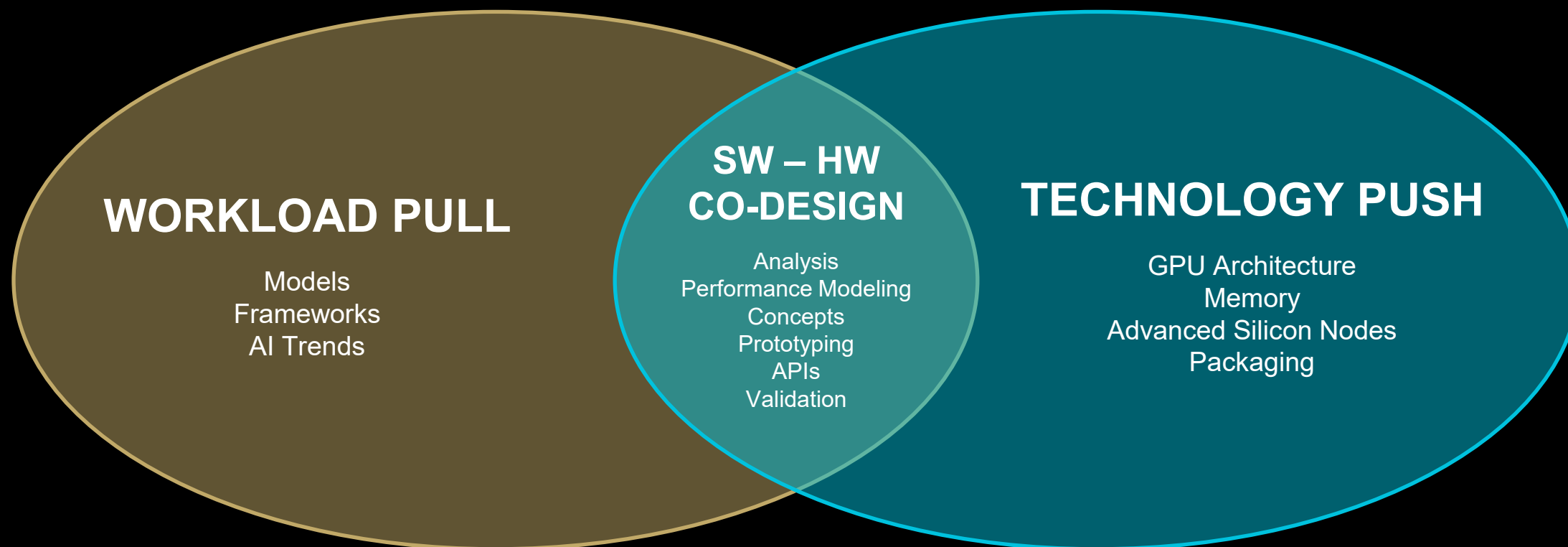
TRAINING		INFERENCE • PREFILL	INFERENCE • DECODE
WORK UNIT	Many full sequences	All prompt tokens together	One new token / sequence / step
COMPUTE	Forward + backward + optimizer	Large GEMMs + full attention	Small GEMMs/GEMVs + KV attention
MODEL STATE	Weights + activations + gradients	Read weights; build KV cache	Read weights + growing KV cache
SYSTEM GOAL	Time-to-train + scaling efficiency	Time to first token + throughput	Time per token + interactivity

MODEL AND SERVICE VARIANTS MODIFY EVERY PHASE

DENSE	every token uses the same MLP weights	MoE	router selects top-k experts → routing + all-to-all	MULTIMODAL	text, image, audio and video become token streams
-------	---------------------------------------	-----	---	------------	---

Hardware co-design begins by choosing the phase, modality, batch size, context length and latency target

Software – Hardware Co-Design at the Intersection



Co-design connects where AI is going with what GPU hardware should become

Software & Workloads

Relentless Focus on Developers

**Feature
Velocity**

Performance Optimization

**Deep Open Ecosystem
Partnerships**

**Richer Out-of-the-box
Experience**



3M+

Models on Hugging Face
Running out-of-the-
box

TOP

10 AI Open-Source Projects
Natively Supported on AMD

>10x

Increase in
OSS Contributions

ROCm Strategy and Evolution



Open Source

Driving Rapid Innovation

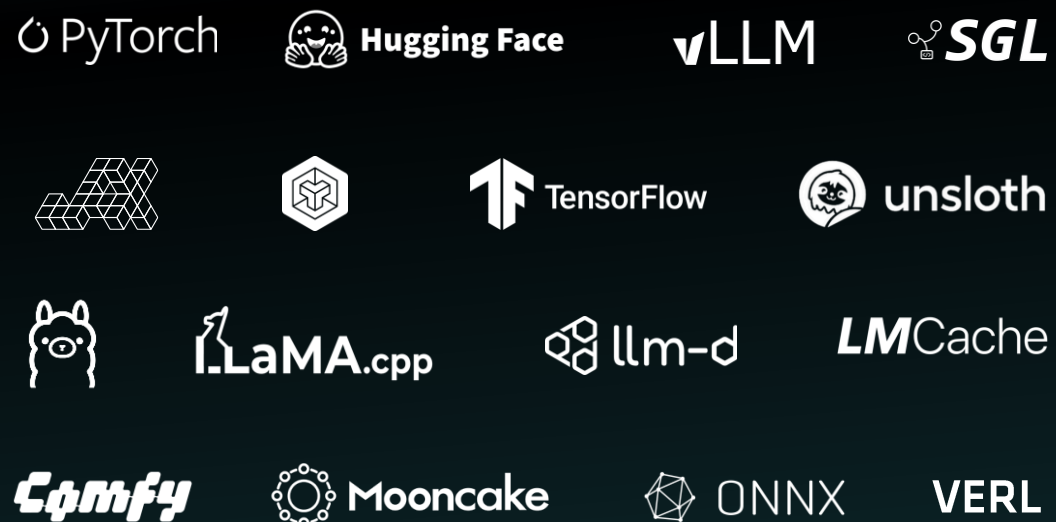


Abstraction

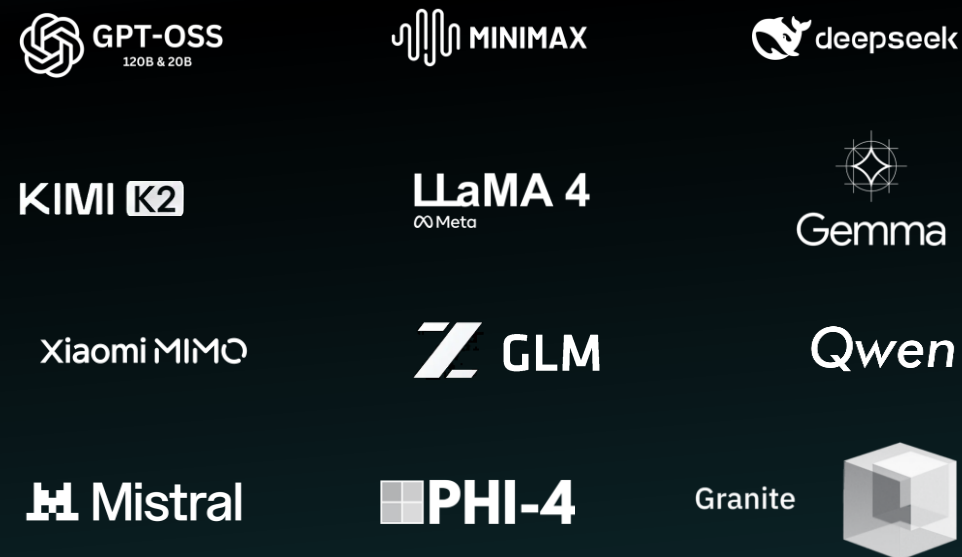
Enabling Higher Productivity

Open Source Strategy Accelerating Progress

OPEN SOURCE COMMUNITY ENABLING AMD



WORLD'S MOST IMPORTANT WORKLOADS RUN ON AMD



Abstraction Makes Deployment Easy on AMD

Program at Every Level. Leveraging Rich Ecosystem.



FRAMEWORKS	Rapid innovation at the highest level	PyTorch · JAX · vLLM · SGLang · TensorFlow · ATOM
PYTHONIC DSLS	Express intent in Python	OpenAI Triton · Gluon · TileLang · FlyDSL
LIBRARIES	Performance with productivity	AITER · hipBLASLt · HipKittens · RCCL · MoRI
LOW-LEVEL PROGRAMMING	Maximum control and performance	HIP C++ · AMDGPU Assembly

Triton vs. Gluon: trading abstraction for control

Shared Python DSL + JIT stack • Triton automates mapping; Gluon exposes it		
GPU MAPPING DECISION	TRITON	GLUON
Tile → lane / warp layout	Compiler selects	Author specifies
Registers + shared memory	Compiler manages	Author orchestrates
Data movement + swizzling	Mostly inferred	Explicitly directed
Async pipeline stages	Compiler schedules	Author constructs
Hardware-specific features	Backend exposes	Kernel can target

Productivity + portability

Control + peak performance

SIDEBAR • CHOOSING THE ABSTRACTION

Why Gluon at all—why not HIP or CUDA?

HIP/CUDA expose everything—the question is how much implementation burden the kernel author should own

TRITON

GLUON

HIP / CUDA

Tile algorithm

Compiler owns mapping

Tile + explicit layout

Author steers mapping

Threads + memory + sync

Author implements mapping

MORE AUTOMATION

MORE IMPLEMENTATION CONTROL

WHY GLUON?

- | | | |
|-----------|--|---|
| 01 | Keep the productive stack | Python DSL, JIT specialization and Triton-style launching/autotuning |
| 02 | Expose performance-critical choices | Layouts, memory movement, swizzling, synchronization and pipelines |
| 03 | Shorten the co-design loop | Prototype hardware-aware mappings without writing a full low-level kernel |

HIP / CUDA STILL WIN WHEN...

- The feature is not exposed by the DSL/compiler
- Exact thread-, instruction- or ABI-level control is required
- A mature production kernel justifies the engineering cost

Gluon is the expert middle layer: substantially more control than Triton, substantially less machinery than HIP/CUDA

Inference shifts the GPU bottleneck by phase and modality

PHASE / MODALITY DOMINANT KERNELS	HBM CAPACITY	HBM BANDWIDTH	SCALE-UP FABRIC	RELATIVE STRESS • FRONTIER-SCALE INFERENCE		
				TENSOR ALUs	VECTOR ALUs	TRANSCENDENTAL ALUs
LLM PREFILL full-sequence attention • large GEMMs	●●○○	●●●○	●●●○	●●●●	●●○○	●●●○
LLM DECODE KV-cache reads • small GEMMs / GEMVs	●●●●	●●●●	●●●●	●●○○	●●●○	●●●○
MoE ROUTING + EXPERTS top-k + all-to-all • expert GEMMs	●●●●	●●●○	●●●●	●●●●	●●●○	●●○○
VISION / DIFFUSION conv / GEMM • denoise • normalization	●●○○	●●●○	●●○○	●●●●	●●●○	●●●○
SPEECH / AUDIO streaming conv • feature transforms	●●○○	●●○○	●○○○	●●●○	●●●○	●●○○
SAMPLING / POST-PROCESS softmax • top-k • RNG • transforms	●○○○	●●○○	●○○○	●○○○	●●●●	●●●●

●○○○ LIGHT ●●○○ MODERATE ●●●○ HIGH ●●●● DOMINANT

Transcendentals: exp / log / reciprocal / rsqrt in softmax, normalization, activations, and sampling

Stress is workload-dependent: batch, sequence length, sparsity, model placement, caching, and precision can move every rating.

No single peak metric predicts inference performance—balance and programmability determine utilization



MODEL ↔ INFRASTRUCTURE CO-EVOLUTION

Frontier inference is becoming a pod-scale workload

MODEL SCALE + STRUCTURE

REPRESENTATIVE PUBLIC MILESTONES

2017 TRANSFORMER 65M dense single-model parallelism	2020 GPT-3 175B dense tensor + pipeline parallel	2022 PaLM 540B dense thousands of accelerators	2024 DEEPSEEK-V3 671B / 37B active 256 experts • top-8 expert parallelism	2025 KIMI K2 1.04T / 32.6B active 384 experts • top-8 agentic + long context
↓ WEIGHTS + EXPERTS larger footprint → more HBM per GPU	↓ ROUTING + COLLECTIVES EP / TP → faster scale-up bandwidth	↓ REASONING + CONTEXT more tokens + KV → larger GPU domains		

GPU + SCALE-UP DOMAIN

NVIDIA REFERENCE SYSTEMS

2017 V100 32 GB HBM2 0.30 TB/s NVLink 8-GPU DGX	2020 A100 80 GB HBM2e 0.60 TB/s NVLink 8-GPU DGX	2022 H100 80 GB HBM3 0.90 TB/s NVLink 8-GPU DGX	2024–25 GB200 186 GB HBM3e / GPU 1.80 TB/s / GPU 72-GPU NVL	2026* RUBIN 288 GB HBM4 3.60 TB/s NVLink 72-GPU NVL
--	---	--	--	--

* PRELIMINARY

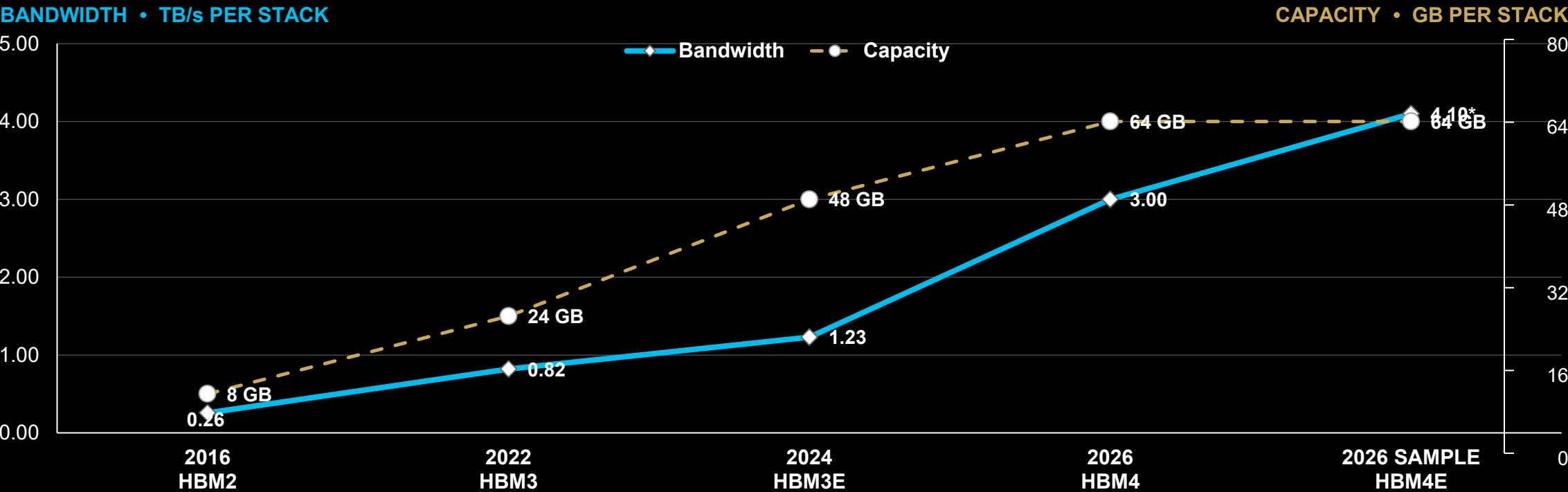
MoE lowers active FLOPs—but raises capacity, routing, and collective-communication demands



Technology & Architecture

MEMORY TECHNOLOGY ROADMAP

HBM bandwidth is scaling faster than stack capacity



KEY DEVICE ATTRIBUTES

* THEORETICAL MAX

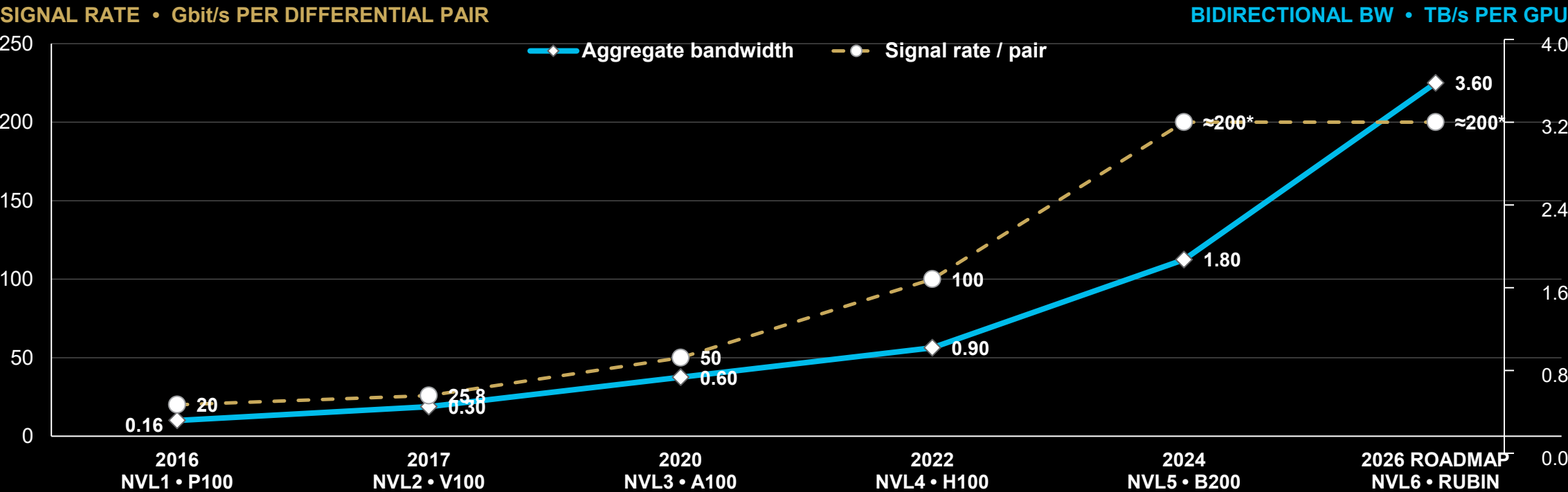
HBM2	HBM3	HBM3E	HBM4	HBM4E
8 Gb die • 8H	16 Gb die • 12H	24 Gb die • 16H	32 Gb die • 16H	32 Gb die • 16H
2.0 Gb/s • 1024b	6.4 Gb/s • 1024b	9.6 Gb/s • 1024b	11.7 tested • 2048b	16 max • 2048b

HBM4 shifts scaling from DRAM alone to memory + logic + package co-design



SCALE-UP INTERCONNECT ROADMAP

Pod bandwidth scales through faster SerDes + more links



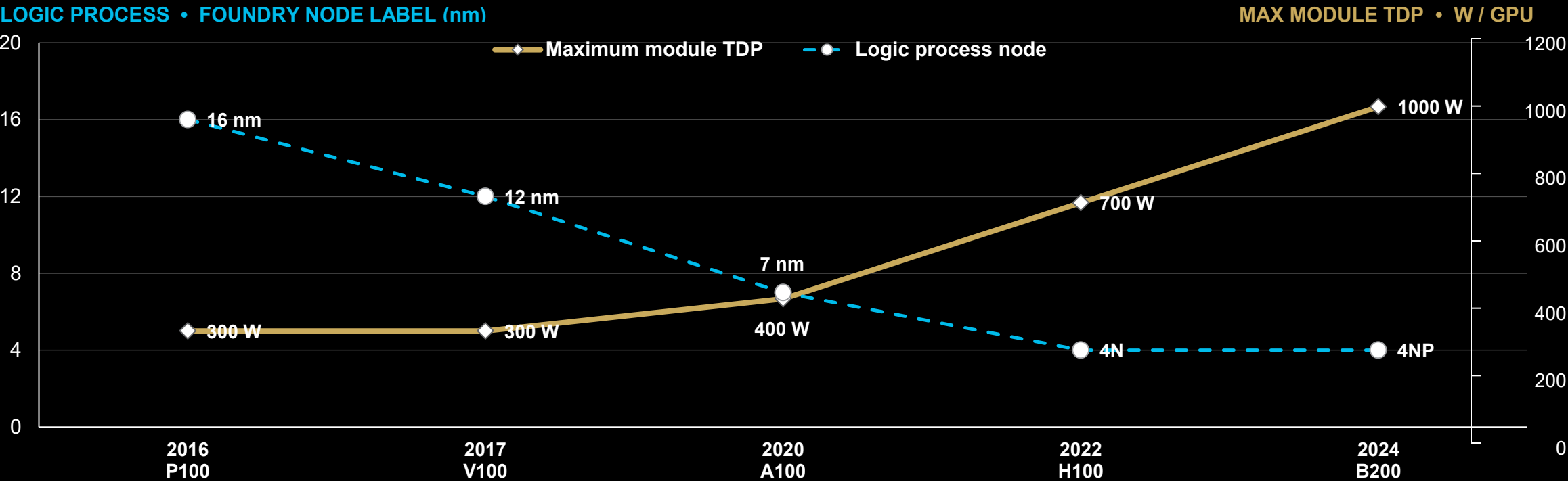
KEY LINK ATTRIBUTES

* IMPLIED; RAW PAIR RATE NOT PUBLISHED

NVL1	NVL2	NVL3	NVL4	NVL5	NVL6
P100 • 4 links	V100 • 6 links	A100 • 12 links	H100 • 18 links	B200 • 18 links	Rubin • 36 links
20G • 8 pairs/dir	25.8G • 8 pairs/dir	50G • 4 pairs/dir	100G • 2 pairs/dir	≈200G* • 2 pairs/dir*	≈200G* implied

NVLink6 doubles bandwidth by doubling links; published per-link bandwidth stays flat

When scaling slows, the GPU “scales-in”



PACKAGING RESPONSE

PACKAGE IS NOW A SCALING VECTOR

RETICLE-SCALE MONOLITHIC

P100 → H100 • 610–826 mm² logic die
GPU + HBM on a full silicon interposer

MULTI-DIE GPU

B200 • 2 reticle-limited dies
208B transistors • 10 TB/s die-to-die

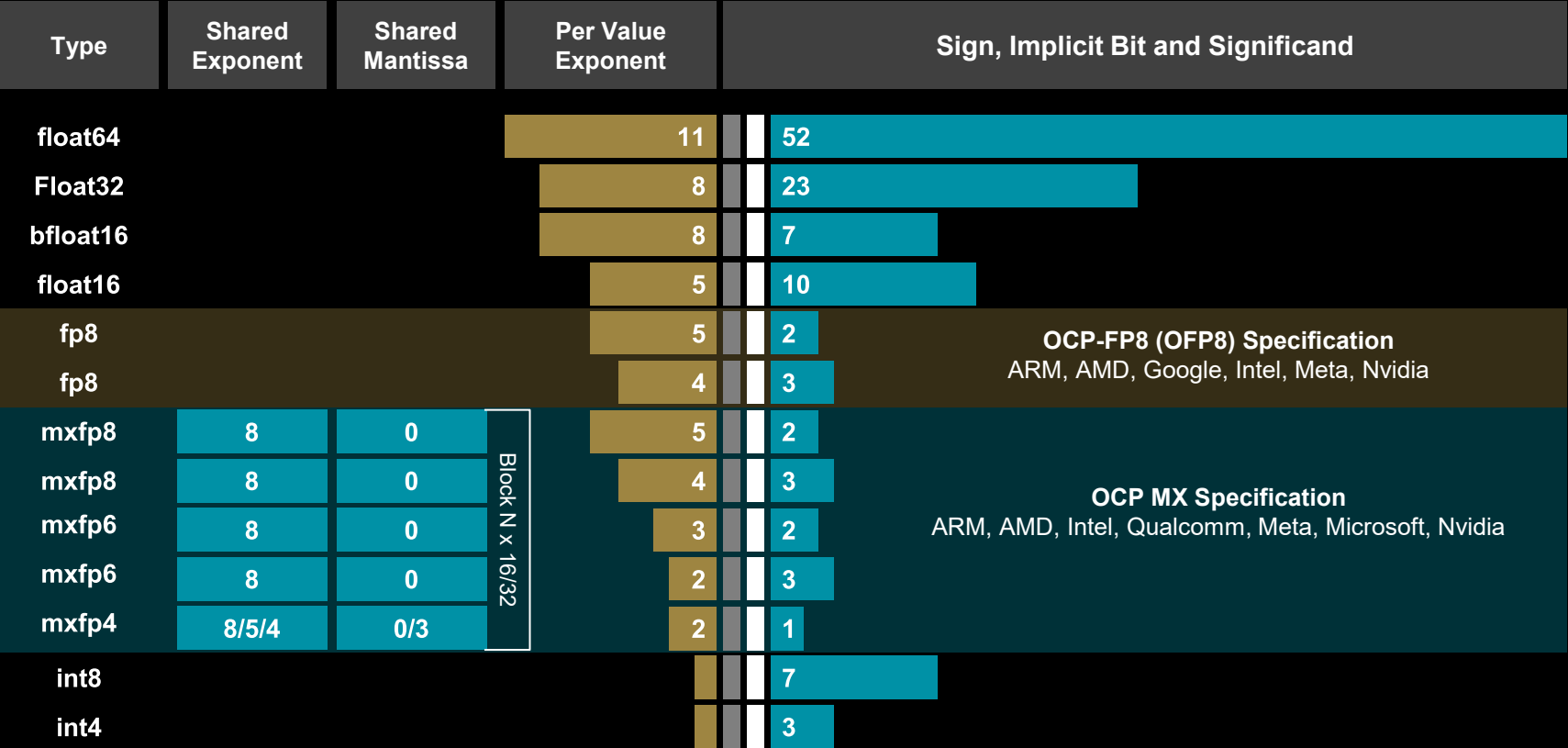
LARGER + 3D PACKAGES

CoWoS-S ≤3.3× → CoWoS-L ≥3.5× reticle
RDL + local Si bridges • SoIC 3D stacking

Slower density and voltage scaling shift performance growth into more silicon, more package—and more power



Optimized Compute For AI



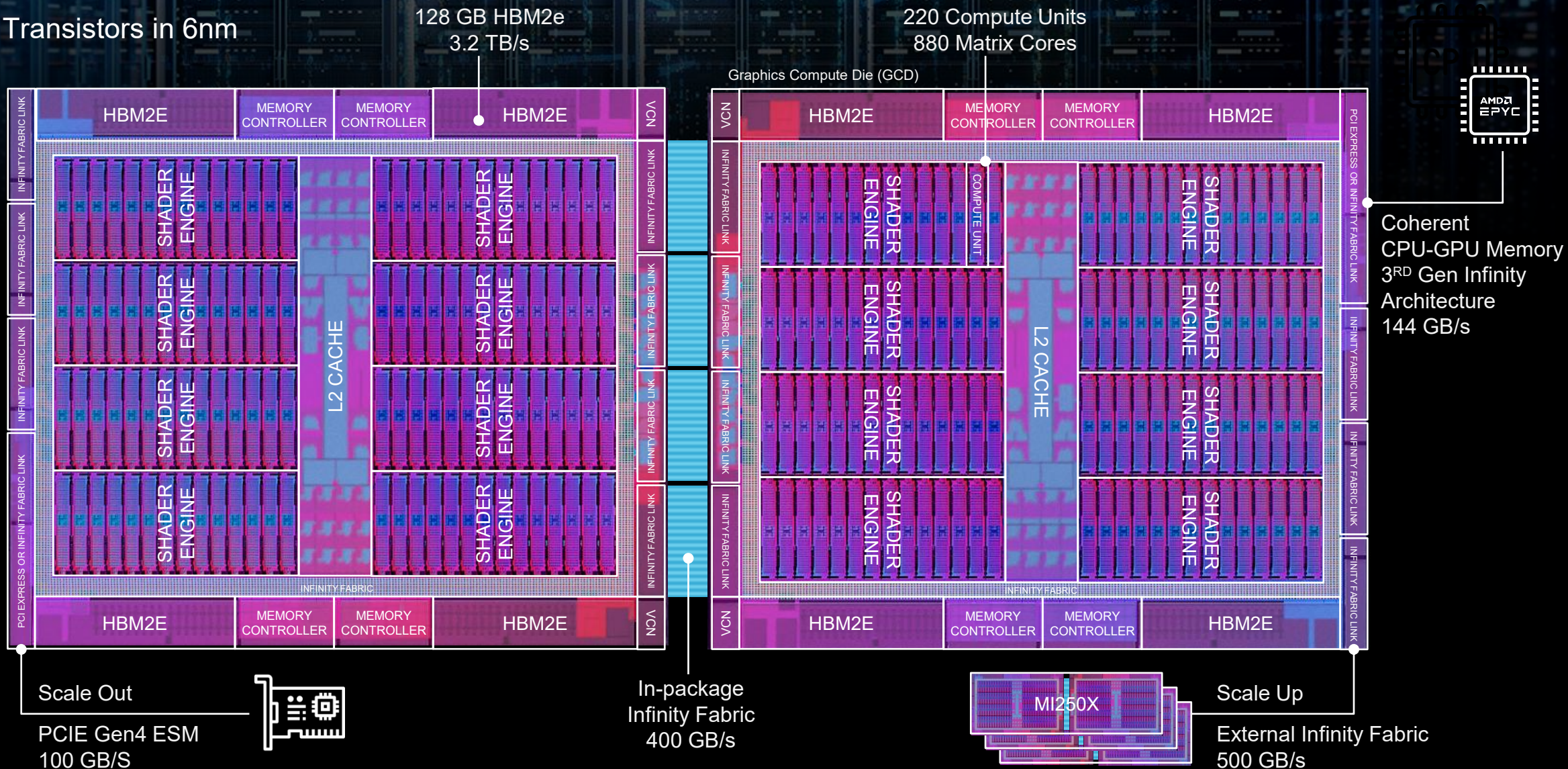
Blocks sharing a common scale factor may now be 16 or 32 elements in size, and a new fractional scale is available for MXFP4 data.

4-bit tensor LUT instructions allow 4-bit memory storage and conversion to 4/6/8-bit MXFP compute formats.

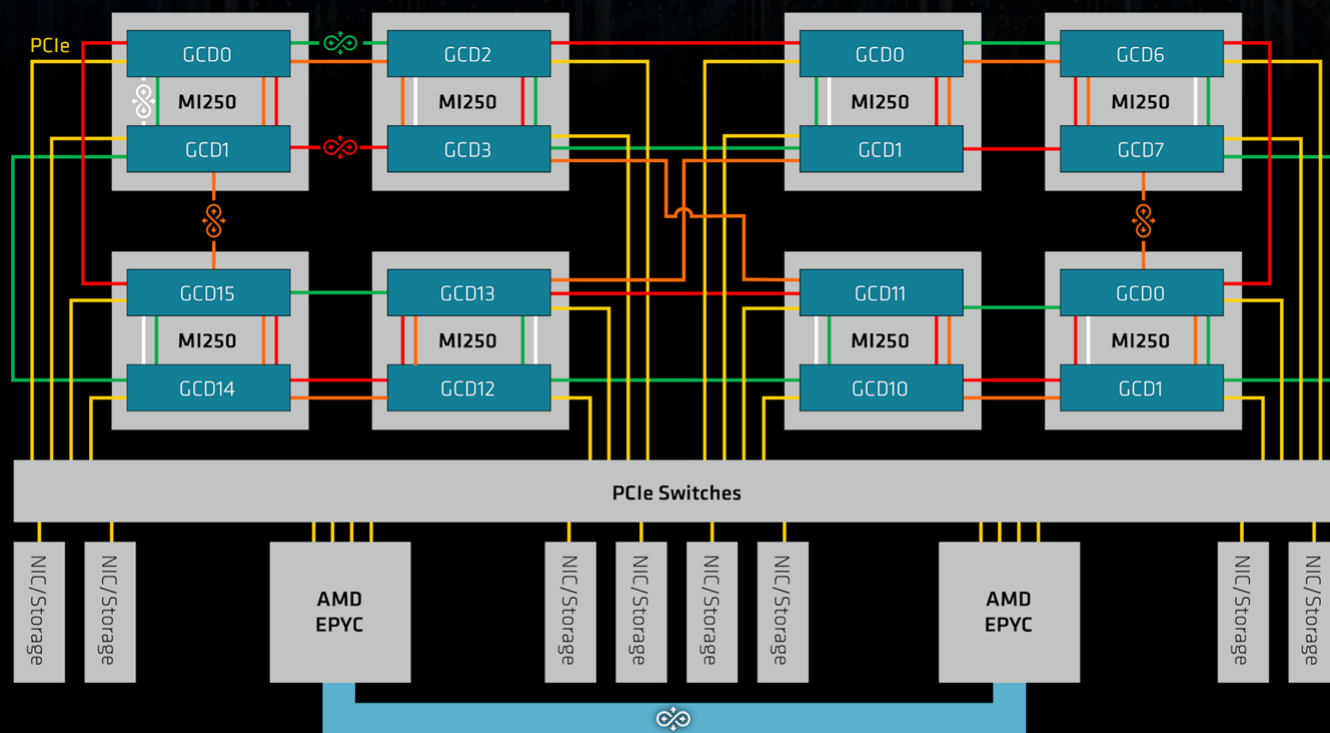
Instinct GPU and System Survey

MI250X MCM

58B Transistors in 6nm



Flagship Machine Learning Topology with AMD Instinct™ MI250 GPUs



- Green, Red, White, and Orange lines are AMD Infinity Fabric™ Links
- Green, Red, and Orange links can create two bi-directional rings across 16 GCD's for All-reduce communication in machine learning networks
- Blue Infinity Fabric Link provides CPU to CPU connection. This is represented by a thick blue line because the number of Infinity Fabric Links between CPUs can vary, depending on system implementation
- Yellow lines are PCIe® Gen4

Two 3rd Gen
AMD EPYC™ Processors

Eight AMD Instinct
MI250 Accelerators

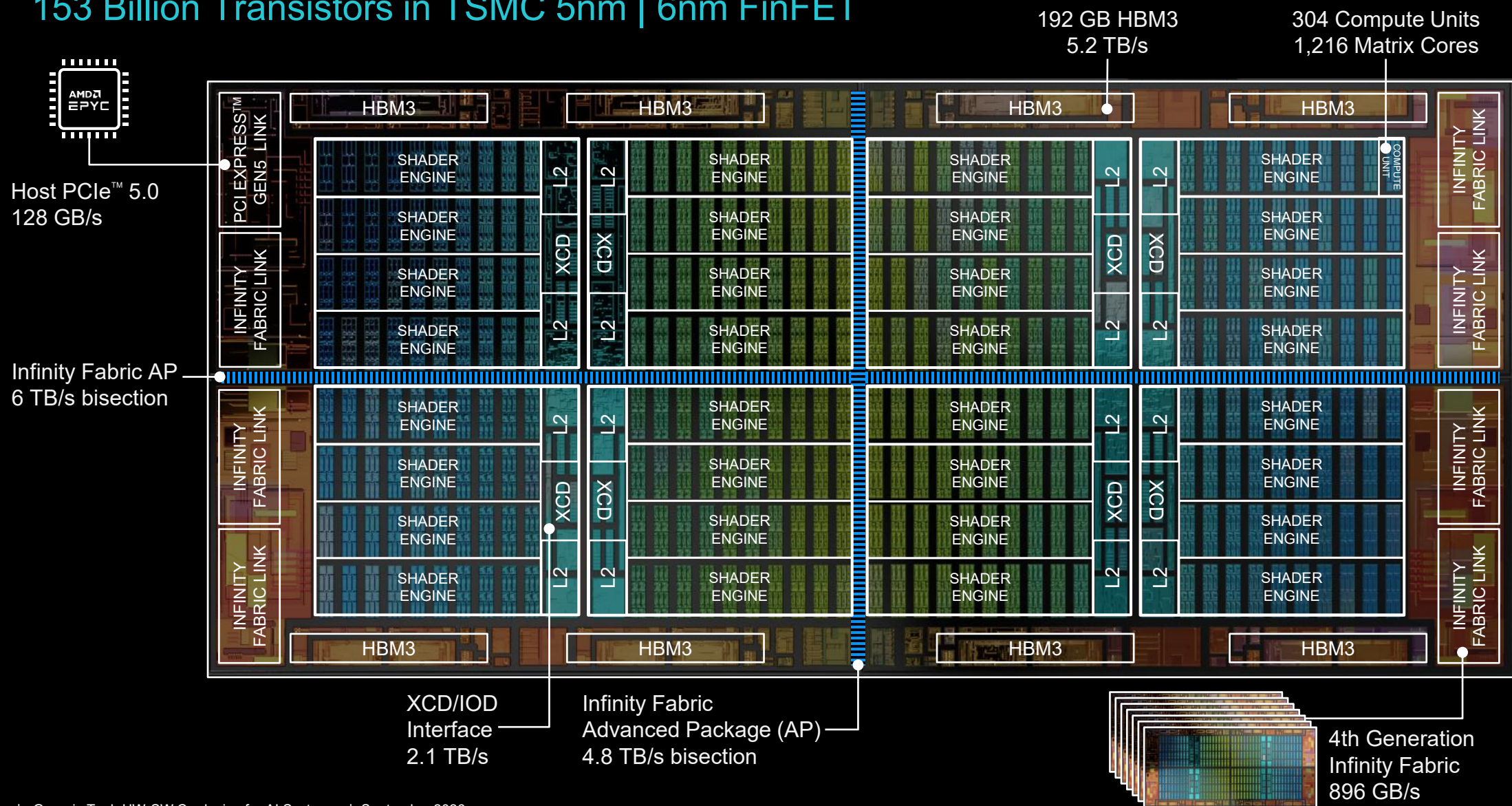
5 GPU-to-GPU
Infinity Fabric Links

2 PCIe® Gen 4
CPU-to-GPU Links Per OAM

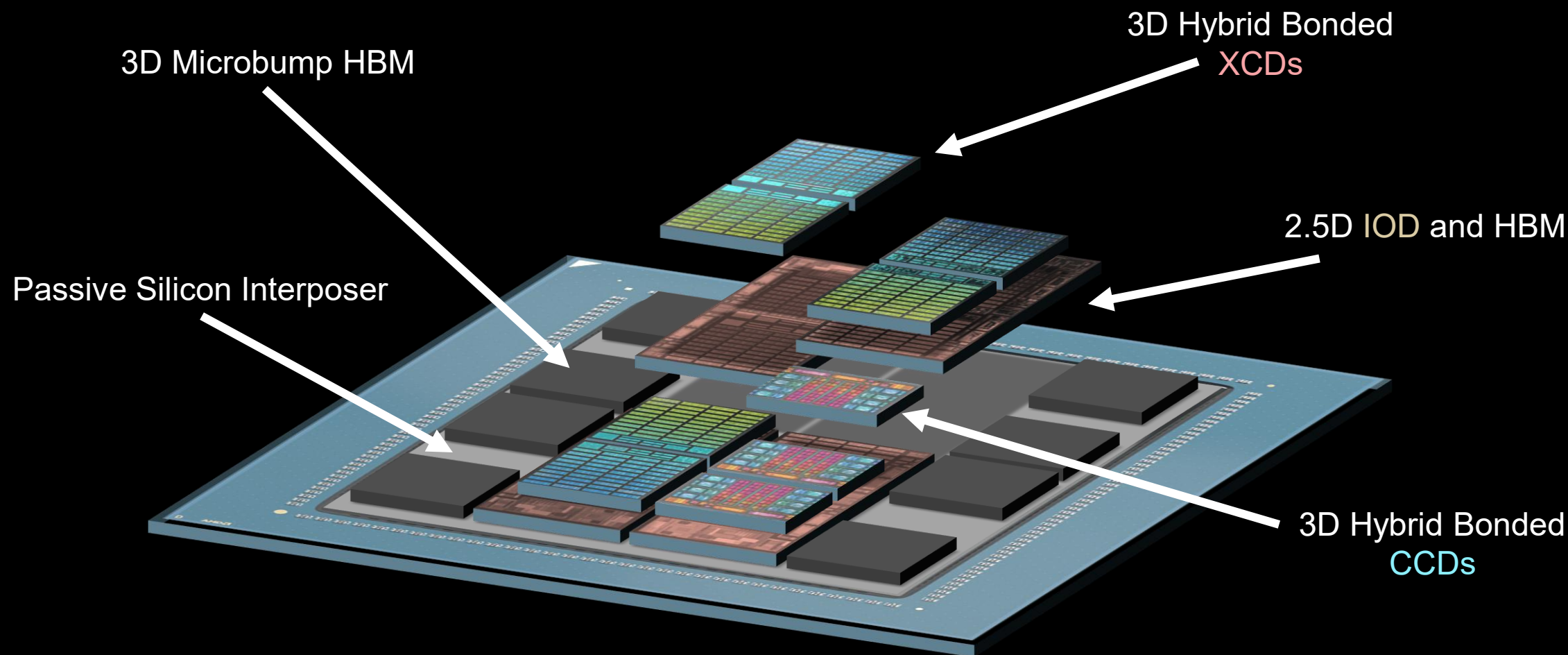
PCIe® Switches
for NIC RDMA

AMD Instinct™ MI300X Multi-chiplet Accelerator

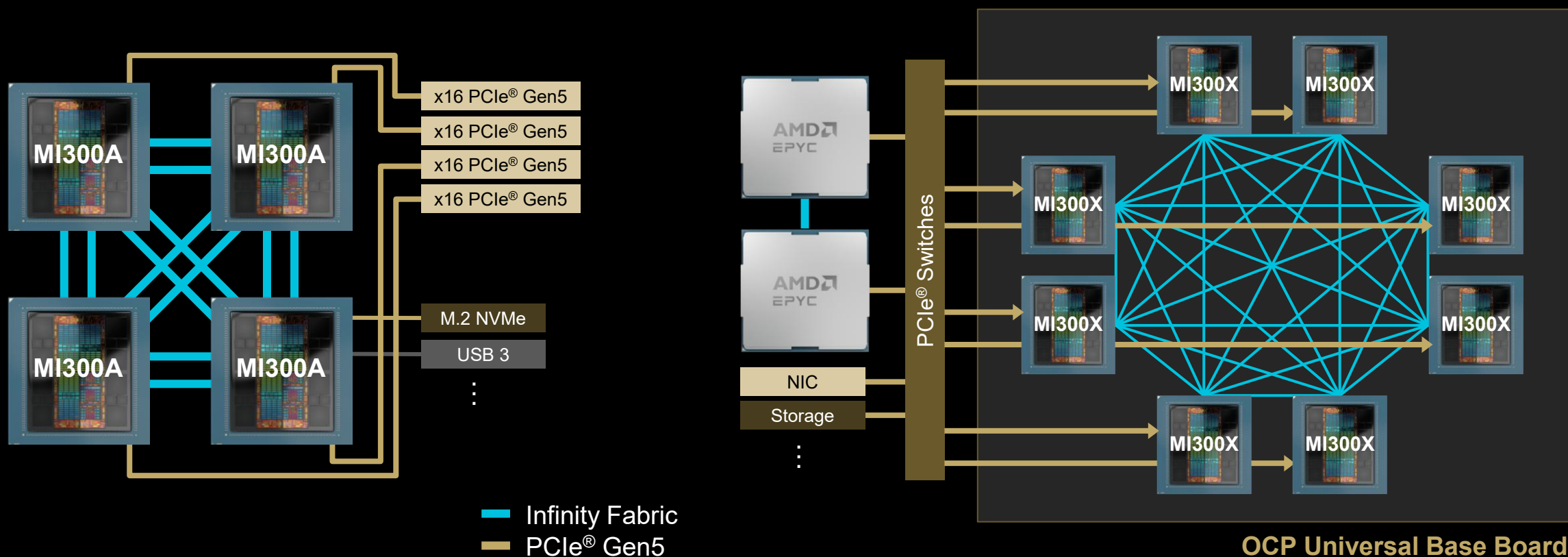
153 Billion Transistors in TSMC 5nm | 6nm FinFET



AMD Instinct™ MI300A Accelerated Processing Unit



Modular I/O Provides Scale-Up Flexibility



- 8 physical stacks with 128 Channel Interface
- 288GB total – 36GB per 12Hi stack at 8Gbps
- 8.0 TB/s total bandwidth

- Supports full bandwidth, flat address space for all chiplets
 - using Infinity Fabric Advanced Package (AP) with 5.5 TB/s bisection
- 2 IOD provide enhanced power efficiency compared to MI300
 - Wider data pipelines enable higher bandwidth at lower frequencies
- 256MB AMD Infinity Cache™

7 - 4th Gen AMD Infinity Fabric™ Links between sockets

1075.2 GB/s bi-directional aggregate

128 GB/s per link



AMD Accelerator Complex Die (XCD)

32 AMD CDNA™ 4 Compute Units (CU) per XCD

Up to 8 XCD per package

256 CU Total

1,024 Matrix Cores

16,384 Stream Processors

2.4 GHz Peak Engine Clock

512 KiB Vector Registers/CU

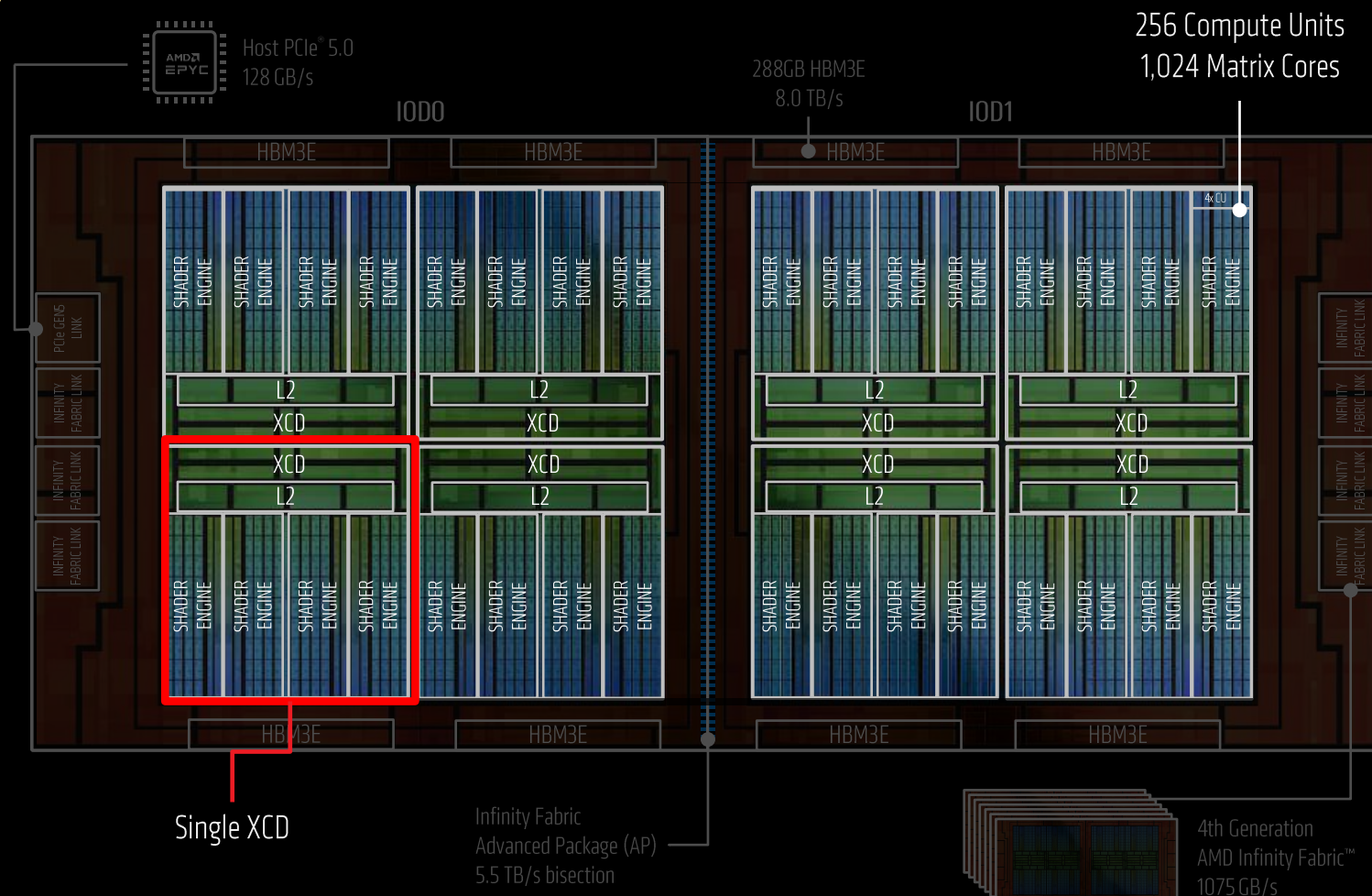
160 KiB Local Data Storage (LDS) @ 537 GB/s/CU

32 KiB of L1 cache per CU

4 MiB Shared L2 cache per XCD

- XCDs have their own GPU host interface using PCIe® Single Root I/O Virtualization (SR-IOV)

- Each XCD
 - can be its own GPU Compute PartitionOR
 - can collaborate with other XCDs forming various sizes of GPU Compute Devices



AMD Instinct™ MI350 Series Node GPU Interconnect

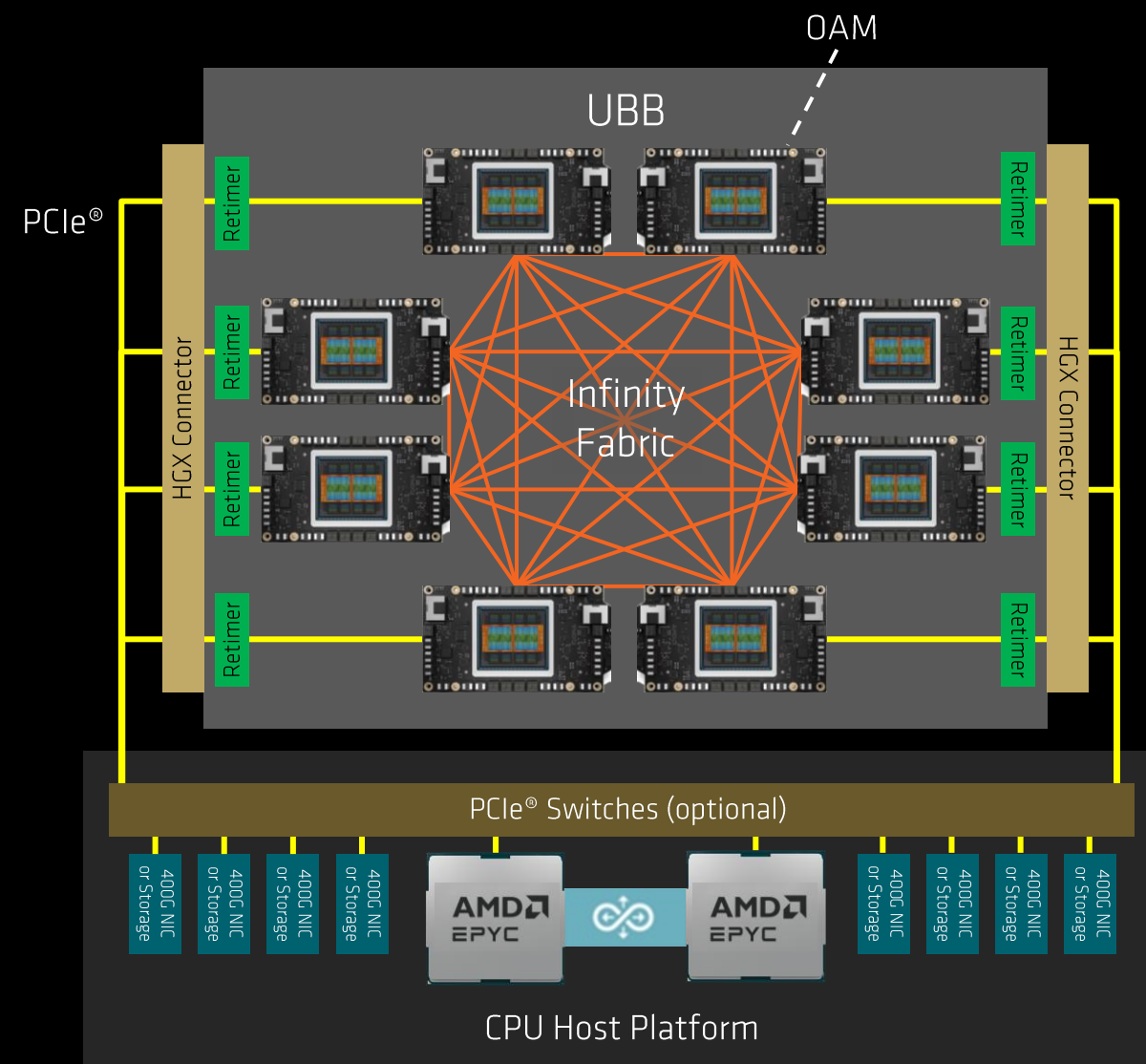
Infinity Platform

PCIe® Gen5 x16 per GPU

- For CPU host server connectivity and I/O
- HGX connectors with PCIe® retimers per link

AMD's Infinity Fabric™

- Direct all-to-all connectivity between 8 GPUs
- Seven bi-directional links @ 154 GB/s
- 20% faster than previous Instinct offering (38.4 vs. 32 Gbps)



AMD Instinct™ MI455X GPU

Enhanced Modular Chiplet Architecture

2x Fabric and Cache Die (N3P)

192 Channel HBM4 Interface

192 MB Global L2 Cache

2x I/O Die (N3P)

2x PCIe Gen 6 or 3x AMD AI-NICs using UALink

Infinity Fabric, 256 GB/s BW (bi-dir)

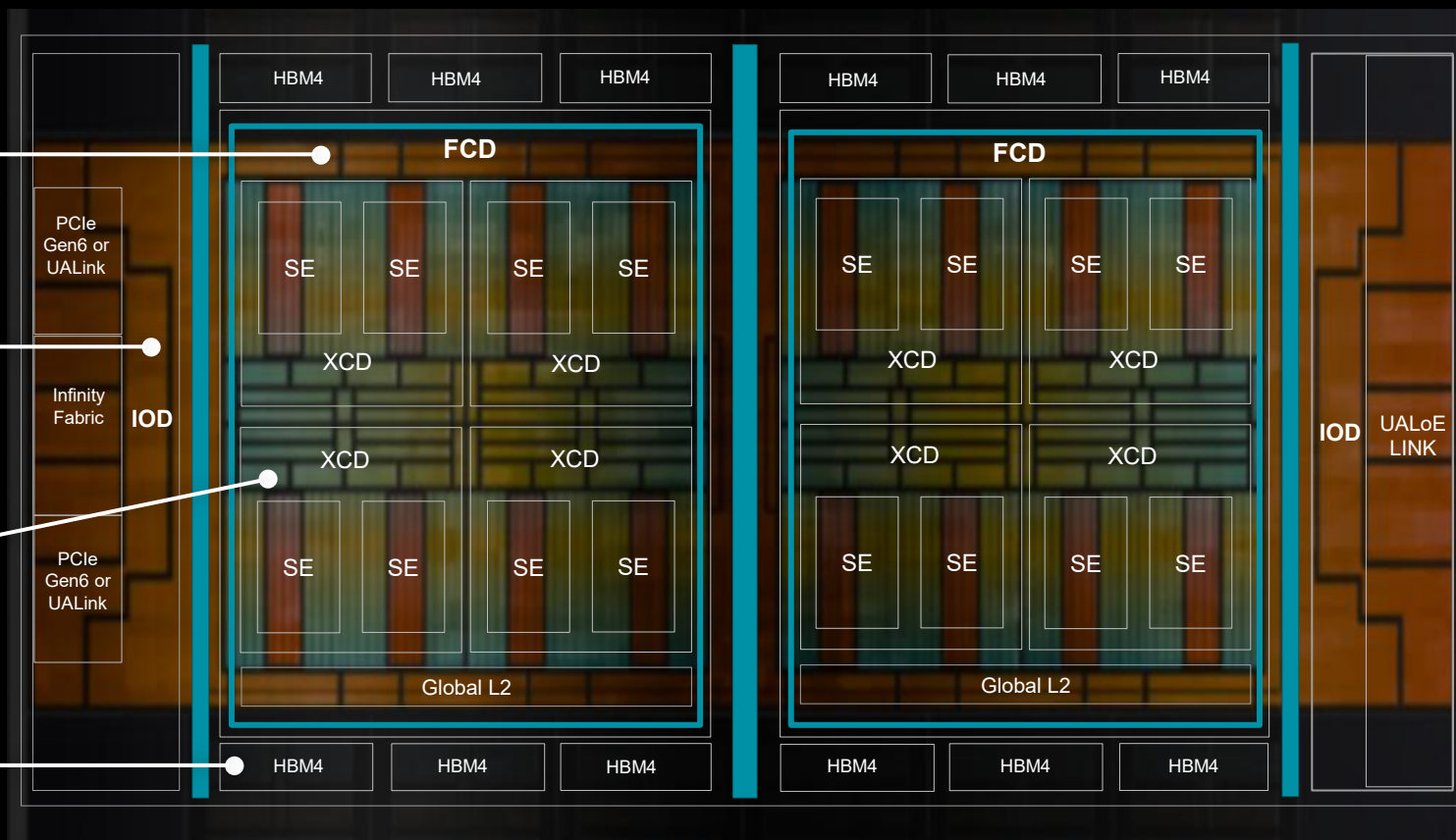
72 UALoE lanes, 3.6 TB/s BW (bi-dir)

8x Accelerator Complex Die (N2)

256 Total Active Work Group Processors

12x HBM4 Memory

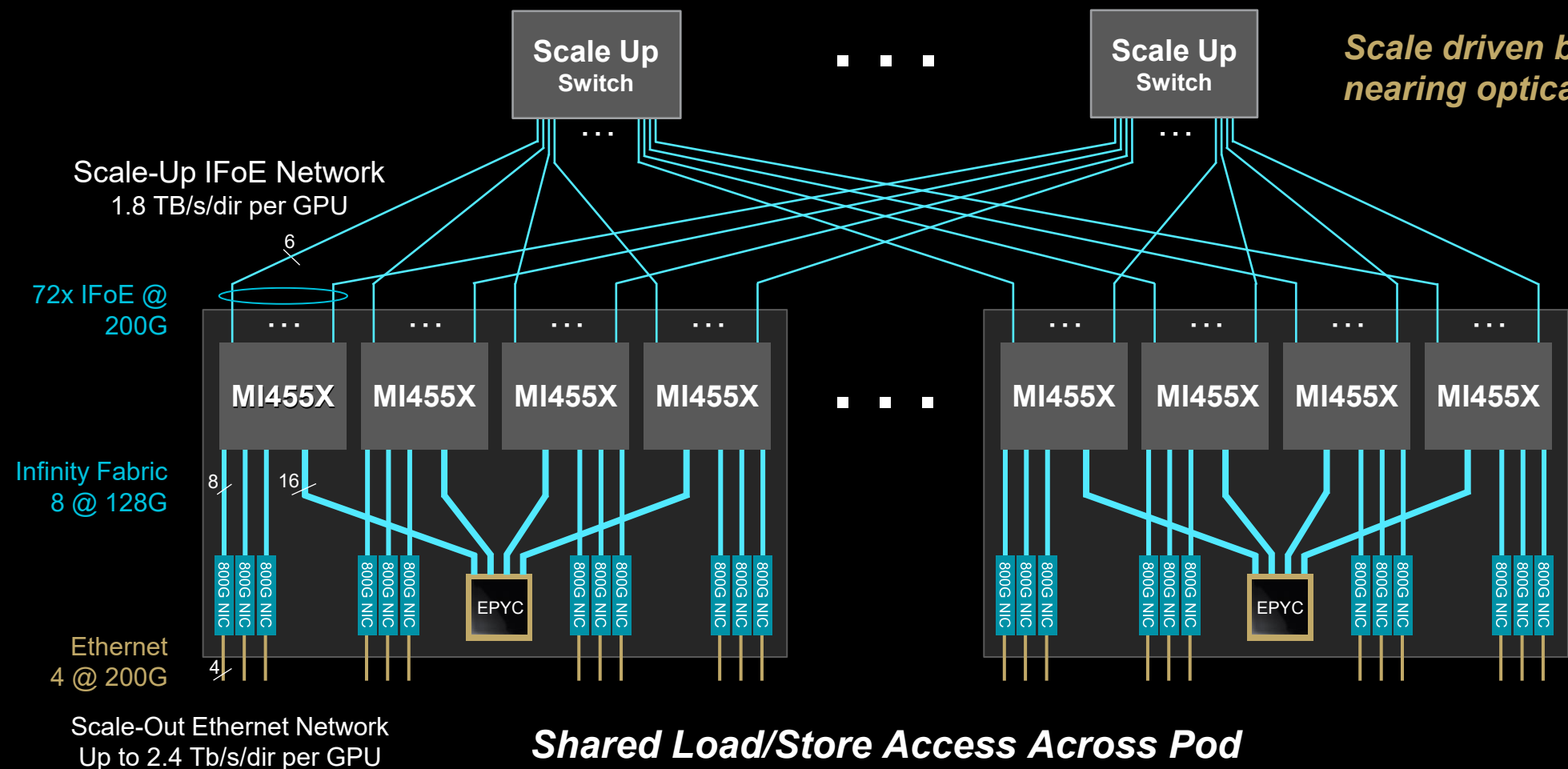
432 GB at 23.3TB/s



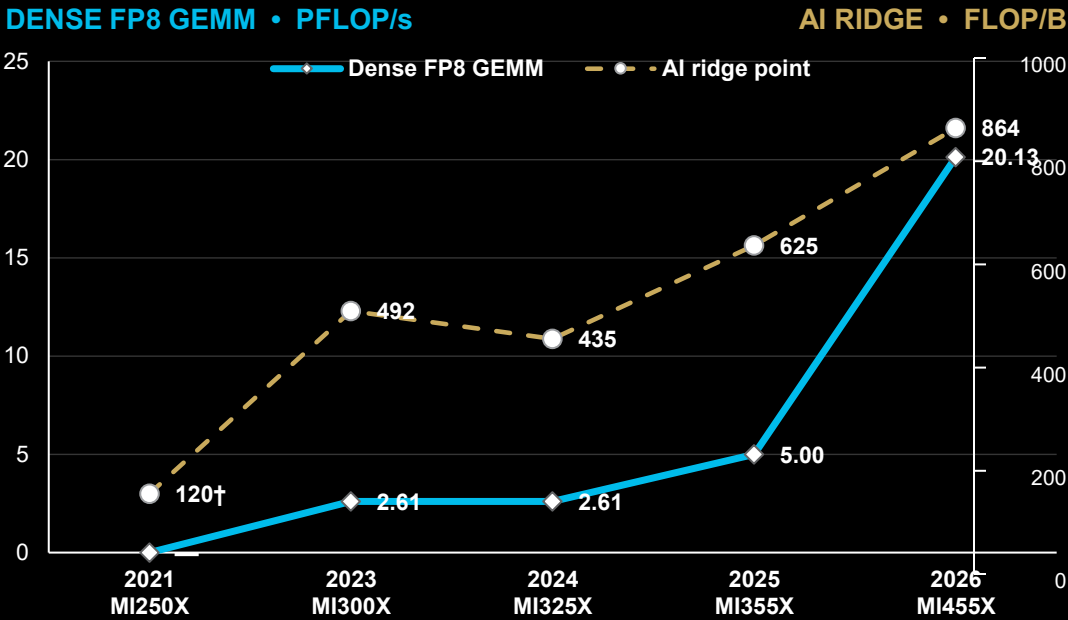
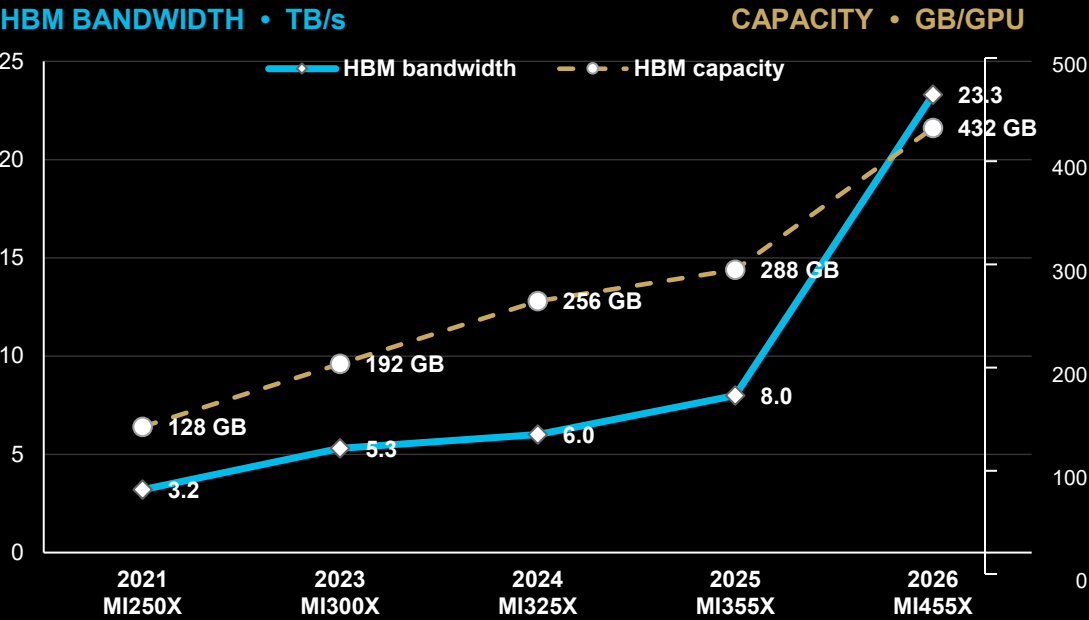
Scale-Up Pod Architecture

Switched topology for maximal communication flexibility

**Scale driven by electrical signaling;
nearing optical transition**

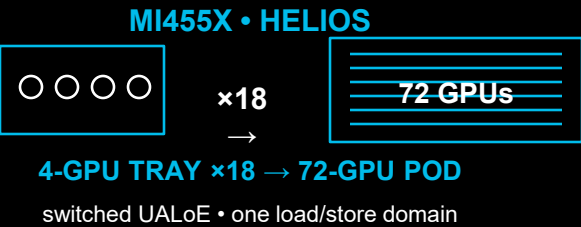
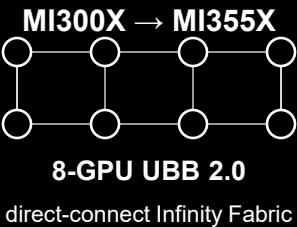
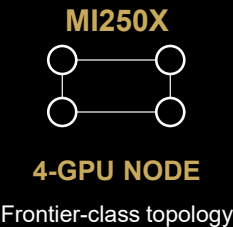


Scaling moves the roofline performance



AI ridge = peak matrix compute ÷ HBM bandwidth • † MI250X uses BF16; FP8 not supported

AI NODE TOPOLOGY



THE SCALE-UP DOMAIN GROWS WITH THE DEVICE

The AI node is now the rack: GPU, HBM, fabric, power, and software must be co-designed

